



Examination 2 (Solution)

Author : Masaki Nishimoto (maspy)

Subtask 1 (S does not contain $\&$ or $|$)

In this subtask, the boolean value the IOI function maps to does not depend on the order of operations. The answer can be determined using the following two pieces of information:

- The number of $!$.
- The multiset of all integers a that are the subject of Rule 1.

The multiset mentioned can be easily obtained by parsing only the parts enclosed in $[]$ brackets. This subtask can be solved in $O(N + Q \log N)$ time complexity.

Subtask 2 ($Q = 1$)

This subtask requires basic handling of parsing. The detailed implementation of the parsing method is omitted here.

Given that the problem statement does not provide a BNF notation, there might be some confusion regarding the correct implementation of rule precedence. If you are unable to implement it correctly, please refer to the following BNF notation:

- $\langle \text{FUNC_1} \rangle ::= '[\ \langle \text{number} \rangle\]'$
- $\langle \text{FUNC_2} \rangle ::= \langle \text{FUNC_1} \rangle \mid '(\ \langle \text{IOI_FUNCTION} \rangle\)'$
- $\langle \text{FUNC_3} \rangle ::= \langle \text{FUNC_2} \rangle \mid '! \ \langle \text{FUNC_3} \rangle'$
- $\langle \text{FUNC_4} \rangle ::= \langle \text{FUNC_3} \rangle \mid \langle \text{FUNC_4} \rangle \ '&' \ \langle \text{FUNC_3} \rangle'$
- $\langle \text{FUNC_5} \rangle ::= \langle \text{FUNC_4} \rangle \mid \langle \text{FUNC_5} \rangle \ '^' \ \langle \text{FUNC_4} \rangle'$
- $\langle \text{FUNC_6} \rangle ::= \langle \text{FUNC_5} \rangle \mid \langle \text{FUNC_6} \rangle \ '| \ \langle \text{FUNC_5} \rangle'$
- $\langle \text{IOI_FUNCTION} \rangle ::= \langle \text{FUNC_6} \rangle$

This subtask can be solved in $O(N)$ time complexity.



Subtask 3 ($N \leq 10,000$)

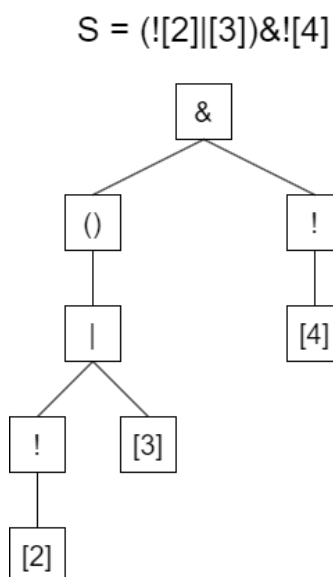
The boolean value X maps to is determined solely by its relationship with the integers subject to Rule 1. Therefore, as long as we calculate the boolean values for the $O(N)$ integers, we can determine the answer for each query in $O(\log N)$ time.

If we perform parsing for each of the $O(N)$ integers, this subtask can be solved in $O(N^2 + Q \log N)$ time complexity.

Additionally, if we can represent the IOI function as a tree (as will be discussed later), the boolean values can be determined in $O(N)$ time. Thus, in this subtask, a solution that spends $O(N^2)$ time on parsing is acceptable. For instance, one approach could involve spending $O(N)$ time each time to identify the application point of the highest precedence rule.

Subtask 4 (S does not contain $!$ or $\wedge$)

In the following subtasks, it is assumed that the syntax of the given IOI function has been correctly parsed. It is convenient to represent the result of the syntax parsing in a tree, as shown below.



Under the constraints of Subtask 4, the IOI function exhibits the following monotonicity: there exists a value t such that integers less than t map to False, and integers greater than or equal to t map to True.

By computing this t progressively from the leaves of the tree, this subtask can be solved in $O(N + Q)$ time. Alternatively, we can consider a solution that leverages binary search, focusing on this monotonicity.



Subtask 5 (One child of vertices corresponding to & or | is a leaf)

Consider computing the IOI function's information progressively from the leaves of the tree. Let n denote the number of leaves in a subtree. The mapping of the IOI function is constant within each interval defined by $O(n)$ boundary integers.

Thus, we can maintain the IOI function's information using the following:

- The boolean value x_0 that $X = 0$ maps to.
- The set change consisting of all integers x at which the boolean value changes, i.e., the boundaries where $X = x - 1$ and $X = x$ map to different boolean values.

Rule 3 involves flipping x_0 . For Rules 4 and 6, if one child of a node is not a leaf, we need to perform appropriate element deletions from the set change and handle the boundary conditions. Rule 5 involves performing the XOR operation between two sets.

Among these, only Rule 5 may present computational challenges with a straightforward implementation. However, by implementing a method where we move elements one by one from the smaller set to the larger set, known as the weighted union heuristic, the total number of element transfers across the entire tree becomes $O(\log N)$. Therefore, we can perform the computations for the entire tree in $O(N \log^2 N)$ time.

With the above approach, this subtask can be solved in $O(N \log^2 N + Q \log N)$ time complexity.

Subtask 6 and 7

To optimize the computation for not only Rule 5 but also Rules 4 and 6, we can apply the weighted union heuristic. By appropriately maintaining the IOI function's information, we aim to achieve the following: flipping the boolean values, setting them to `True`, or setting them to `False` over a specified interval.

By enumerating all the relevant values at a particular tree vertex and applying coordinate compression, we can perform the necessary operations using a (dual) segment tree in $O(\log n)$ time. We ensure that for vertices inheriting the same data structure through the weighted union heuristic, the same set of values is computed. This is analogous to performing computations per heavy path as in the Heavy-Light Decomposition (HLD). For more details, you can search for "DSU on Tree".

Alternatively, we can use a data structure known as a dynamic segment tree.

By implementing the above approach correctly, this problem can be solved in $O(N \log^2 N + Q \log N)$ time complexity.



Additional Information

Apart from the methods discussed above, another possible approach involves incrementing X and sequentially determining the boolean values that the IOI function maps to. In this case, it suffices to quickly compute some results from tree DP each time the boolean values of the leaves are updated.

A data structure often used for such dynamic tree DP calculations is the Static Top Tree, which can also be applied to this problem. Additionally, given that the tree in this problem is a binary tree, HLD combined with segment trees can effectively handle dynamic tree DP updates.



Heat Stroke (Solution)

Author: Hirotaka Yoneda (square1001)

0 Introduction

In this problem, we need to calculate how many patients will be sent by helicopter in the worst case. It should be noted that, compared to problems to "optimize something", which is much more usual in competitive programming, this problem is to compute "worst case", so we need to think a bit differently.

This problem contains a wide levels of subtasks, which are expected to be tackled by beginners to world-class competitive programmers. Needless to say the importance of partial points in OI-style contests, make sure to get all subtasks that are within your skill level^{*1}.

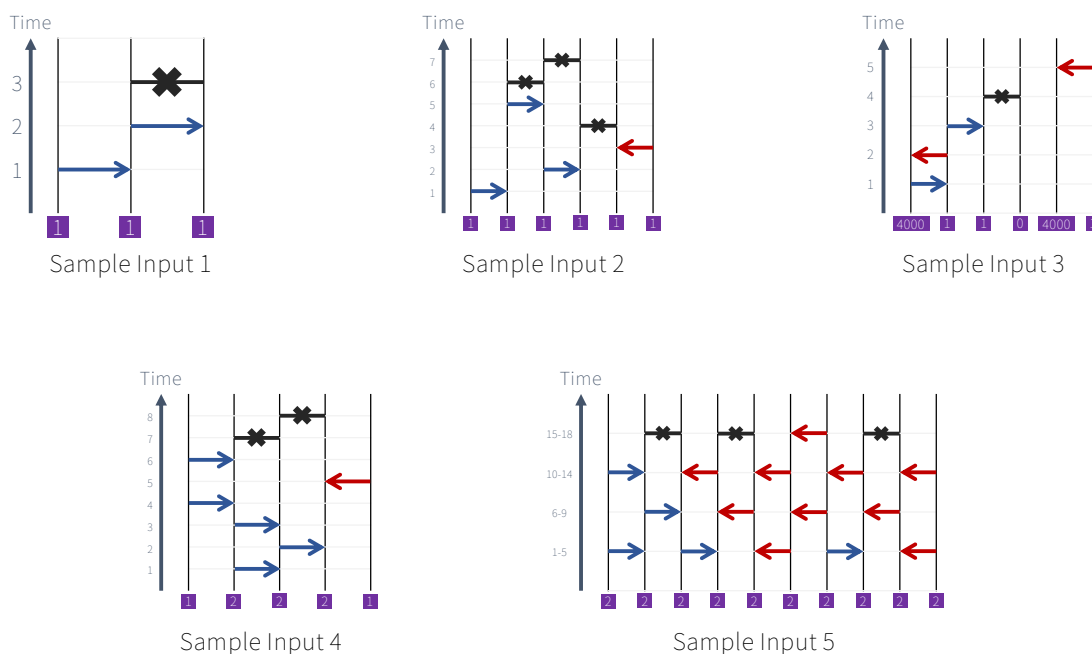


Figure 1 The worst scenario for Sample Inputs 1-5 (Number in purple square is the capacities of hospitals; For Sample Input 2, 3, 5, other "worst scenarios" exist). How many of them did you solve?

^{*1} For example, if you are aiming for IOI Gold Medal, solving subtasks 1-6 (totals 80 points) is a "nice" goal.



1 Subtask 1 ($A_1 \leq A_2 \leq \dots \leq A_N$)

When $A_1 \leq A_2 \leq \dots \leq A_N$, it is worst to send patients rightward in the case of "both hospitals are open". The reason is that, when all patients of road x are sent to hospitals, all not-yet-sent patients are in road $x + 1$ or more right, which means: at this moment, the less vacant the hospital $x + 1$ is, the worse. Let's simulate this "worst strategy" and count the number of patients sent by helicopter.

2 Subtask 2 ($L \leq 18, N \leq 18, C_i = 1$)

We consider brute force, which means to search all possible scenarios of sending patients to hospitals. One method is to brute force all 2^N ways of assignment of, for each patient, "when left/right hospitals are both open, which direction (left or right) to send". Then, we can compute the worst case in $O(2^N(L + N))$ time, which pass Subtask 2.

3 Subtask 3 ($L \leq 18, N \leq 100, C_i = 1$)

When $L = 18, N = 100$, is there are cases that there are exactly 2^{100} different scenarios? The answer is no. When L is smaller than N , the number is far less than 2^N . Let's explain the reason. When sending a patient, there are 2 choices of how to send, only when left/right hospitals are both open (otherwise there is only 1 choice). Since the total capacity of hospitals in the island is L , the event of "choose from 2 choices" occurs at most $L - 1$ times. Therefore, at most 2^{L-1} scenarios exist^{*2}.

Using **depth first search (DFS)**, we can brute force all possible scenarios. We can solve this problem in $O(2^L(L + N))$ time, which pass Subtask 3.

(Continued on next page.)

^{*2} In fact, there is no case to have exactly 2^{L-1} scenarios (when $L \geq 3$). We can prove that the maximum number of scenarios is $O(1.62^L)$. The proof is left for skilled readers.



4 Subtask 4 ($L \leq 100, N \leq 100, C_i = 1$)

Interpret as an Optimization Problem

The setting of this problem is to "find the worst case", which is rather unusual. However, when we consider the problem like the following, we can think like "Among those that satisfies XX, calculate the maximum value of YY", which is a common pattern in competitive programming problems.

Interpretation of Problem Consider the assignment of the way of sending (left/right/helicopter) for each patient (there are 3^N such ways). Among those that is "a valid scenario", we want to find the maximum number of patients sent by helicopter.

The Condition of "Validness"

When we want to solve such problems by Dynamic Programming (DP), we first consider a good characterization of "validness of scenario". Figure 2 shows the 3 ways of assignment of left/right/helicopter, but Example 2 and 3 are invalid. Let's consider the reason.

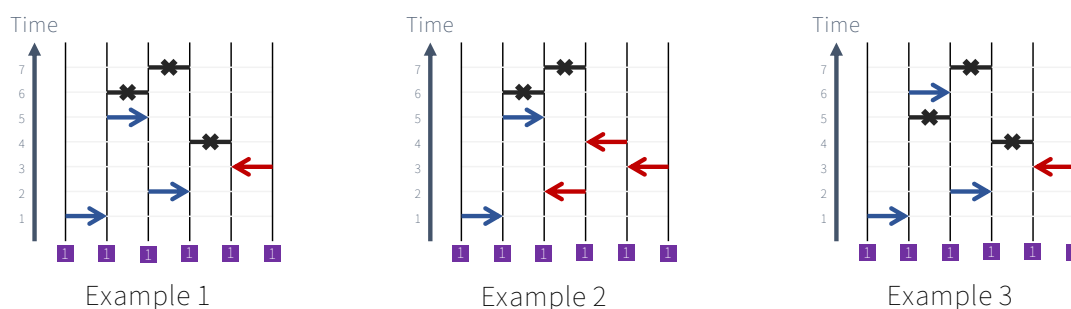


Figure 2 Three ways of assignment of left/right/helicopter in Sample Input 2 (only Example 1 is valid)

- The violation in Example 2 is that, when hospital 3 is full, a patient on road 2 is sent to there.
- The violation in Example 3 is that, when hospital 3 is open, a patient on road 2 is sent by helicopter.

Let's define to say that "the violation at hospital 3 is happening"^{*3}. It should be noted that, when checking

^{*3} The cases of violation at hospital x is (1) when hospital x is full, a patient on road $x - 1$ is sent to right (2) when hospital x is full, a patient on road x is sent to left (3) when hospital x is open, a patient on road $x - 1$, x is sent by helicopter.



the violation at hospital 3, we don't use any information on patients of roads 1, 4, 5. To generalize, checking the violation at hospital x only requires the information of (left/right/helicopter for) patients on roads $x - 1$ and x .

Strengthening the Constraints

In the constraint of $C_i = 1$, when there are 3 or more patients in a single road, the 3rd or later patients are surely sent by helicopter. Thus, erasing all such patients, we can solve the problem with case of at most 2 patients per road. In some competitive programming problems, the technique of "strengthening the constraints" like this works.

Do Use Dynamic Programming

Let's put things together and consider the DP solution. We consider deciding left/right/helicopter from the patients of the leftmost road. Since checking violation at hospital i only requires the information of roads $i - 1$ and i , we can consider the following DP:

$dp[i][F]$ When left/right/helicopter for patients on roads $1, 2, \dots, i$ are already decided, the assignment of left/right/helicopter for patients on road i is expressed as F , and there are no violation at hospitals $1, 2, \dots, i$, the maximum number of patients that are already sent by helicopter.

State Transition The condition that can be transited from state of $dp[i - 1][F']$ to state of $dp[i][F]$ is that, there are no violation at hospital i when the assignment of roads $i - 1, i$ is F', F , respectively.

Since there are at most 2 patients per roads, there is at most $3^2 = 9 (= O(1))$ states of F . Therefore, the DP is done in $O(L)$ time.

(Continued on next page.)



5 Subtasks 5, 6 ($L \leq 600, N \leq 600$)

The DP solution explained Subtask 4 section requires, when C_i is not necessarily 1, requires exponential number of states for F . However, we only need much limited information to check violation at hospital i . In fact, the following two are enough to record:

- j : The number of patients on road i that is sent rightward to hospital $i + 1$.
- t : The ID of the last patient when the hospital $i + 1$ became full^{*4}.

We consider the DP that calculate $dp[i][j][t]$, the maximum number of patients sent by helicopter in this state. The important point is that, t is determined after deciding road $i + 1$, so at the state of $dp[i][j][t]$ **we make reservation to decide road $i + 1$ that "the ID of the last patient when hospital $i + 1$ became full" will match to t** . Then, the computation of all $dp[i][?][?]$ (using $dp[i - 1][?][?]$) takes $O(N^2 Z_{i-1} Z_i)$ time, where Z_i is the number of patients on road i . Since $Z_1 + Z_2 + \dots + Z_{L-1} = N$, we can solve this problem in $O(N^4)$ time.

However, the condition of state transition from $dp[i - 1][j'][t']$ to $dp[i][j][t]$ is complicated, so now let's consider how to make implementation easier. Instead of setting t to be the exact time that hospital $i + 1$ became full, consider setting t as "any time between the time hospital $i + 1$ became full and the time of first helicopter on roads $i, i + 1$ " (the exception is when the hospital $i + 1$ is still open at last, then we can set $t = N + 1$).

Condition of State Transition Let b the number of patients on roads i among patients $1, 2, \dots, \max(t, t')$:

1. The number of patients on road i to go rightward j ($0 \leq j \leq Z_i$) is inside the following range:
 - Case $t' < t$: (the number of patients on road i among patients $t' + 1, \dots, t$) $\leq j \leq b$
 - Case $t \geq t'$: $0 \leq j \leq$ (the number of patients on road i among patients $1, \dots, t$)
2. The number of patients sent to hospital i is "correct":
 - Case $t' \neq N + 1$: $j' + (b - j) = C_i$
 - Case $t' = N + 1$: $j' + (b - j) \leq C_i$

Looking closely to the condition 2., for each (i, j, t) , there are only $O(N)$ valid choices of (j', t') . Therefore, the computation of $dp[i][?][?]$ (using $dp[i - 1][?][?]$) takes $O(N^2 Z_i)$ time, which means we can solve this problem in $O(N^3)$ time.

^{*4} If the hospital $i + 1$ is still open at last, set $t = N + 1$.



6 Subtasks 7, 8

Consider further speeding up the DP solution of Subtask 6. First, as a technique of DP speedup, divide the DP transition into the following 3 cases and solve each cases separately, for not to require "if" condition.

1. Case of $t' < t$
2. Case of $t' \geq t$ ($t' \neq N + 1$)
3. Case of $t' = N + 1$

In fact, for all 3 cases, if we calculate DP values of (j, t) in an appropriate order, we can use **cumulative max** and calculate $dp[i][j][t]$ in $O(1)$ time. It should be noted that the "direction" of using cumulative max is completely different for cases 1., 2., and 3.

Then, the computation of $dp[i][?][?]$ (using $dp[i - 1][?][?]$) takes $O(NZ_i)$ time, which means we can solve this problem in $O(N^2)$ time.

7 Remark

Thank you for reading this review. We uploaded C++ solutions of Subtask 5 and 8.



Library 3 (Solution)

Author : Sora Todaka

Subtask 1 : $N!$ query

Query the all possible $N!$ ways of arrangement of the books. We get 0 only if the queried arrangement matches the correct arrangement.

Subtask 2 : N^2 query

Let $A_1, A_2, \dots, A_N$ be the queried arrangement, and $B_1, B_2, \dots, B_N$ be the correct arrangement. Let $A_{P_1}, A_{P_2}, \dots, A_{P_N}$ be the same as $B_1, B_2, \dots, B_N$. In other words, $(P_1, P_2, \dots, P_N)$ is the permutation that permute queried arrangement into correct arrangement. The number of cycles in a permutation is defined as the number of cycles formed when directed edges are placed from vertex i to vertex P_i out of N vertices.

The value we can get when we send a query is $N - (\text{number of cycles in permutation } (P_1, P_2, \dots, P_N))$. From here on, we will proceed with the assumption that we can get the number of cycles in permutation $(P_1, P_2, \dots, P_N)$ by sending a query.

The change of cycles when we swap the books of two places is as follows.

- If the two places belong to the same cycle, the cycle is split into two cycles so that the two places belong to different cycles.
- If the two places belong to different cycles, the cycles combine to form one cycle.

If we query an arrangement and the arrangement that books of two places are swapped, either one arrangement is that the two places belong to different cycles. Among all pairs of two places, repeat to query the arrangement that books of two places are swapped and get the arrangement that the two places belong to different cycles. Then we can get the arrangement that all pair of two places belong to different cycles. That is the correct arrangement.



Subtask 3 : $N \log N$ query

Assume that we know the arrangement that places $1, 2, \dots, i$ belong to all different cycles. In this case, we can get the arrangement that places $1, 2, \dots, i, i + 1$ belong to all different cycles by $\log(i + 1)$ query.

From assumptions, place $i + 1$ belongs to the same cycle as at most one place out of places $1, 2, \dots, i$. Therefore if we know which place belongs to the same cycle as place $i + 1$, we can get the arrangement that places $1, 2, \dots, i, i + 1$ belong to all different cycles by swapping books of that place and place $i + 1$.

For $j (\leq i)$, we can know whether there is a place that belongs to the same cycle as place $i + 1$ among places $1, 2, \dots, j$ by following query.

query the arrangement that applied the following swapping

$swap_1(\text{place } 1, \text{ place } 2)$

$swap_2(\text{place } 2, \text{ place } 3)$

$\vdots$

$swap_{j-1}(\text{place } j - 1, \text{ place } j)$

$swap_j(\text{place } j, \text{ place } i + 1)$

In this arrangement, all cycles that places $1, 2, \dots, j$ belong to combines into one cycle by $swap_1, \dots, swap_{j-1}$. We can know whether place $i + 1$ belongs to this combined single cycle by $swap_j$.

We can know which place belongs to the same cycle as place $i + 1$ in $\log(i + 1)$ queries by using binary search which find the boundary j where "whether there is a place that belongs to the same cycle as place $i + 1$ among places $1, 2, \dots, j$ " changes. We can know the correct arrangement in total $N \log N$ queries by finding arrangement that places $1, 2, \dots, i, i + 1$ belong to all different cycles for $i = 1, 2, \dots, N - 1$ in order.